DHAR

Bare Metal. Absolute Control.

VOLUME I — VISION & SPECIFICATION

SPEC0

"One language, from source text to silicon, owned end to end."

DOCUMENT	Spec0 — Overview & Vision
VERSION	1.3
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume I
AMENDED BY	Amendments A-1 and A-2
AUTHOR & PROJECT LEAD	Abdullahi Baba Garba

Contents

Philosophy & Naming 3

1. Executive Summary 3

2. The Problem 4

2.1 Technological Dependence 4

2.2 The Missing “Holy Grail” 4

3. The Dhar Thesis 5

4. Architecture Overview 6

4.1 Targets 6

5. Use Cases & Target Domains 7

5.1 The direction 7

5.2 Breadth kept: the modules 7

5.3 Principles and gates 8

6. Language Preview 8

7. Positioning: How Dhar Compares 9

8. The Five Stages 10

9. Reading the Rest of the Canon 11

Philosophy & Naming

Dhar comes from the pre-Islamic Arabic *dahr*: time in its most unbounded sense — a long age, eternity, fate. Classical Arabic lexicons and the Quranic corpus confirm this as the word's core meaning. In pre-Islamic Arabian poetry, dahr was not merely a measurement of duration but an autonomous force — the thing that outlasts kings, wealth, and strength alike, answerable to nothing else. The concept was significant enough that Islamic tradition addresses it directly: a hadith recorded in both Sahih al-Bukhari (4826) and Sahih Muslim (2246) has Allah say, "the son of Adam curses time [dahr], though I am time" — read by classical commentators, al-Nawawi among them, as a correction of the pre-Islamic habit of blaming misfortune on time itself, not as a claim that time is divine. Dahr remains, in every source, time — total, autonomous, and enduring past any single moment or ruler.

That is the inheritance this project draws on, stated plainly rather than overclaimed: independence, robustness, and endurance are not dahr's dictionary definition, they are this Canon's chosen extension of it — the same way most technical names build meaning onto a word rather than translating it literally. A systems language and toolchain meant to outlast the hardware it targets and the individual engineers who build it aspires to the same quality dahr was always used to describe: something whole, self-standing, and built to last. Dahr's other defining quality — encompassing every fortune and misfortune, the entire span of a life rather than one register of it — is where this Canon locates Dhar's multi-paradigm ambition (§2.2 below): a language equally at home in a kernel driver and a web backend, in strict static typing and dynamic scripting, claims a similar totality.

Kun, the name given to Dhar's package manager (Stage-delta, Volume II), comes from the Arabic *kun*, "be" — the imperative at the center of the Quranic phrase *kun fa-yakun*, "be, and it is", appearing eight times in the Quran (2:117; 3:47, 59; 6:73; 16:40; 19:35; 36:82; 40:68) to describe creation by divine command. A package manager's core promise has the same shape, at a vastly smaller scale: declare what should exist in a project, and it comes into being — resolved, fetched, and ready — without being assembled by hand. The metaphor is offered in that spirit, not as a claim of equivalence with what the phrase means in its own context.

1. Executive Summary

Dhar is a from-scratch systems programming language and compiler toolchain, designed around a single, deliberately ambitious target: performance at least at C's level, Java-class safety, Python-class readability *and* Python-class dynamism where it's wanted, genuine multi-paradigm support, and full independence from any external compiler backend or runtime. No LLVM. No JVM. No borrowed interpreter. The compiler is bootstrapped once and then produces its own machine code through its own backend, permanently.

Dhar is a language for **trustworthy computation**. It goes first to cybersecurity, where a mistake is an attack; then to artificial intelligence, where software must run models on modest hardware; then to quantum computing, where cryptography must outlast a quantum computer and quantum programs must be described and simulated. Every other domain is kept, as modules on the same core (§5.1, §5.2; Amendment A-1).

Dhar's motivation is Nigerian and African technological independence at the systems-software layer — the layer beneath applications, where compilers, linkers, and runtimes live, and where almost no African-built alternative currently exists. This document is Spec0: the front door to the Canon. It states the problem, the technical thesis, the

architecture, the target use cases, a taste of the language, and how Dhar is meant to sit relative to the languages that already exist. The full grammar and ratified keyword semantics live in Spec1. The stage-by-stage engineering plan lives in Volume II.

2. The Problem

2.1 Technological Dependence

Most of the software infrastructure the world runs on — compilers, operating-system kernels, virtual machines, cryptographic libraries, the toolchains that turn source code into running systems — was designed, and continues to be maintained, outside Africa. That is not a criticism of any of it: C, Rust, Java, Python, and the compilers behind them are mature tools built by large communities over decades. But it means that when African institutions build the software critical infrastructure runs on — banking cores, telecom switches, government platforms, defence systems — they build it on foundations they did not design, cannot unilaterally audit at the lowest level, and do not set the roadmap for.

Dhar starts from a simple observation: Nigeria, and Africa more broadly, does not currently have a home-grown, from-scratch systems programming language and toolchain — something built at the same layer as C, Rust, or Go, by people who can audit, extend, and maintain every layer of it locally. Closing that gap is not only about the language. It is about growing a generation of compiler engineers who understand their tools from the lexer up, building software that can be exported rather than only imported, and giving the institutions that most need to trust their infrastructure a toolchain whose internals are not a black box.

2.2 The Missing “Holy Grail”

The target combination this Canon builds toward is informally called the “Holy Grail” of programming languages: not a guarantee, but a target specific enough to force real trade-off decisions. No mainstream language currently holds all of the following at once:

- **C-class performance** — near-hardware, at least matching C, not merely approaching it: no GC pauses, no interpreter overhead, memory laid out and controlled at compile time.
- **Java-class robustness** — strict typing, with safety mechanisms such as automatic bounds-checking making silent memory corruption structurally difficult, not merely discouraged.
- **Python-class simplicity and dynamism** — code that reads close to plain language, with type inference doing the work a beginner would otherwise do by hand, and a dynamic-typing mode (Spec1 §4.4) available for exploratory, script-like code, the same way Python code can be written without ever declaring a type.
- **Genuine multi-paradigm support** — procedural, functional, object-oriented, logical, and symbolic styles combined intelligently in the same language (Spec1 §10), not bolted on.
- **Full independence** — no LLVM, no JVM, no external interpreter runtime, because the compiler is bootstrapped and produces its own machine code through its own backend.

Each of the five is an aim. The test that says whether it has been met is a gate (§5.3; Amendment A-1 §9), and no statement that it has been met is made without naming the gate (Tempering Dhar §11).

Every existing language trades at least one of these away: C and C++ trade safety and simplicity for performance; Java trades independence and some performance for safety and portability; Python trades performance and low-level control for simplicity; Rust gets close on performance and safety but keeps a steep learning curve and, by default, an LLVM code generator; Zig and Odin get close on independence-in-spirit but have relied on LLVM for code generation (Zig is building back ends of its own). Naming the full combination as the target is what lets every later decision in this Canon be checked against it.

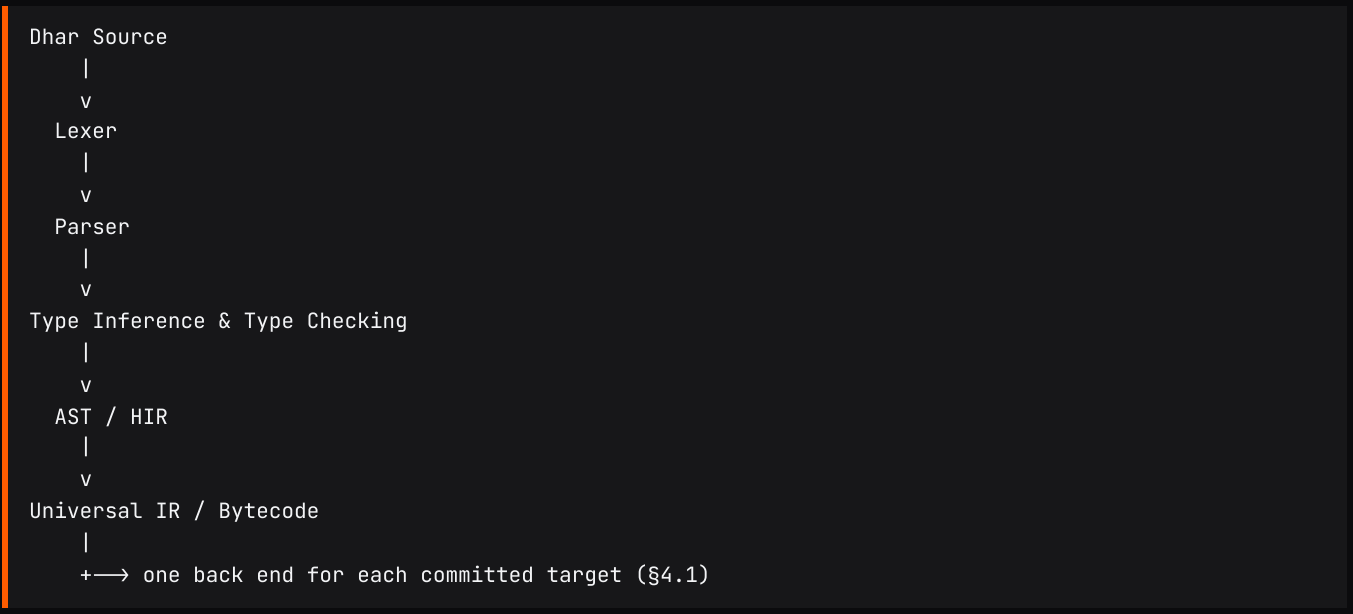
Linux did not win because Unix was rare. It won because it was a complete, independently built, freely extensible foundation that the world could stand on without asking permission. Dhar's ambition at the language-and-toolchain layer is the same kind of foundation, not a niche experiment.

3. The Dhar Thesis

Five linked design commitments answer “how.” Each is expanded into full engineering detail in its corresponding Volume II stage document.

1. **The parser is not written in assembly — the backend is.** Hand-writing a full lexer, parser, and type checker directly in assembly does not scale; parsing complexity would bottleneck progress before the language is usable. Assembly is reserved for what it's good at: a library of code-generation templates — pre-written structures with the hardcoded values stripped out — that a higher-level compiler fills in and emits.
2. **A universal virtual CPU is defined before any real one is targeted.** The same way Python and Java compile to an intermediate bytecode rather than directly to hardware, Dhar defines its own universal instruction layout, not tied to any physical architecture. The compiler only ever targets this universal layout; a small backend module per committed target (§4.1) turns it into real machine instructions. Portability is structural: a new architecture means writing one backend module, never touching the parser or type system. Stage-alpha, the bridge, reaches every committed target without this layer, through one small prelude per target; the Universal IR begins at Stage-beta, whose back ends are held to the output of Stage-alpha's (Amendment A-2 §4, §7).
3. **Manual memory management is traded for compiler-inserted discipline.** Python's simplicity comes from not managing pointers by hand; C's speed comes from having no background garbage collector. Dhar keeps the manual-management-free surface syntax while the compiler automatically inserts allocation and deallocation logic during compilation, following a compile-time ownership model (Rust's approach) combined with defer/arena allocation (Zig's approach) where appropriate.
4. **Safety is baked into the templates, not into a garbage collector.** Every array access is generated with an automatic bounds check, so an invalid index produces a safe, diagnosable crash instead of silent corruption — with no separate runtime safety layer required.
5. **The language is bootstrapped through a disposable bridge, then proven by rebuilding itself.** A minimal bridge compiler, written in a language that already works, compiles just enough of Dhar to build a real compiler written entirely in Dhar. That compiler then compiles itself; if the resulting binary is byte-for-byte identical to the one that produced it (ignoring timestamps), Dhar has proven it no longer needs the bridge, which is retired to a historical artifact rather than kept as a permanent dependency. The bridge is frozen and archived, never extended, and never a dependency of a Dhar build; it stays reproducible because the two-compiler check (Amendment A-1, S4; Tempering Dhar §5.3) needs a second, independent compiler to compare against.

4. Architecture Overview



The load-bearing rule: nothing above the Universal IR line is allowed to know which hardware it is ultimately targeting. The lexer, parser, type checker, and AST construction only ever need to understand Dhar's own syntax and semantics. All architecture-specific knowledge is pushed into backend modules that consume the same IR and each know how to lower it to one target. Stage-alpha's code generation is direct, through the template-emitter approach of the Thesis, with no intermediate representation (Amendment A-2 §4); Stage-beta generates the Universal IR, and Stage-gamma (Optimizing Dhar) is where optimization passes operate on it.

4.1 Targets

Dhar is built for every processor and operating system the Canon names, and the list of them is in exactly one place: Amendment A-2 §3.1. This document links to it and does not restate it.

1. **One back end per target.** From Stage-beta on, each target is a small back end on the Universal IR and a description table of the processor (Amendment A-1, R4). Stage-alpha reaches the same targets without an intermediate representation, through one prelude per target, and its output is the reference that the Stage-beta back ends are compared against (Amendment A-2 §4, §7).
2. **Complete before the next stage.** Stage-alpha is complete only when every committed target has passed gate G1 on a real system, and Stage-beta begins then (Amendment A-2 §2, §6).
3. **An unsupported target is a fatal error.** The compiler never falls back to another target and never produces output for a target it has not been built and tested for (Amendment A-2 §2).
4. **A target changes the encoding of a program, not its meaning** (Amendment A-2 §8), so one source program does the same observable thing on every committed target.
5. **Claims name their gate.** A statement that Dhar supports a target names the gate and the run that passed it (Amendment A-2 §6; Tempering Dhar §11).

5. Use Cases & Target Domains

Dhar serves many domains. §5.1 says in which order the work on them is done, §5.2 lists them, and §5.3 states the principles and the tests that hold across all of them.

5.1 The direction

Dhar is a language for **trustworthy computation**, and the work goes in this order: **cybersecurity** first, then **artificial intelligence**, then **quantum computing**. Each step uses what the one before it built: tensors rest on the checked views and ownership of the first, and post-quantum cryptography is written with the hash functions of the first. The order is a statement about what is built and tested first, never about what Dhar is for: every other domain of §5.2 is served by modules on the same core.

No work on the three tracks begins before Stage-beta compiles itself (Amendment A-1 §7). Until then the work is Stage-alpha, Stage-beta and the design requirements that the tracks place on the compiler's intermediate representation (Amendment A-1 §8).

5.2 Breadth kept: the modules

Nothing is removed (Amendment A-1, P6). Every domain below is delivered as modules on Kun and the standard library (Stage-delta §2.1), built on the same core and held to the same safety floor, so that Dhar stays modular and robust and able to do what its users need. The three tracks come first, then the rest.

- **Cybersecurity & security tooling** — memory-safety guarantees plus low-level access (`peek` / `poke` , `raw[N]`) in one language, with secrets that the compiler checks (Spec1 §14.5).
- **Defence & critical national infrastructure** — a toolchain whose entire pipeline, from lexer to machine code, can be independently audited end to end, with no externally controlled dependency in the critical path. Specified in full in Tempering Dhar (Volume III).
- **Artificial intelligence** — inference with memory use known before the run: numeric types for models, fixed-shape tensors and arenas (Spec1 §4.7, §14.6), with no garbage collector and no outside runtime.
- **Quantum computing** — cryptography that survives a quantum computer, and the description and simulation of quantum programs.
- **General systems programming** — OS components, drivers, and other code currently written in C or Rust.
- **Hardware & embedded programming** — direct memory operations and fixed-size buffers as language primitives.
- **Backend & server software** — C-class performance without a GC pause.
- **Web** — compiled backends, with WASI as a first-class backend target, and the in-browser playground (Tooling Dhar §5).
- **Mobile** — through the ARM64 targets; which mobile systems are candidates, and which is excluded, is stated in Amendment A-2 §3.2 and §3.3.
- **Data** workloads — performance-critical numeric code without leaving a readable syntax.
- **Blockchain & Web3** — deterministic execution and fine-grained resource control.

5.3 Principles and gates

Six principles hold across every domain above. They are stated in Amendment A-1 §3; this is a summary, and the amendment is the home of each.

- **P1. Independence.** The core language, compiler and standard library depend on no outside compiler, runtime or library; outside implementations are test oracles, never dependencies.
- **P2. Safety by construction.** Every index is checked, ownership is enforced at compile time, and there is no garbage collector; the few places where safety is given up are explicit and findable by search.
- **P3. Every claim has a gate.** A statement about Dhar's security, speed or correctness is allowed only if a test against a named reference passes.
- **P4. A small, auditable path from source to executable**, with reproducible output and a recorded list of what went into every build.
- **P5. Nothing retrofitted.** What the tracks need from the compiler is designed into the first intermediate representation.
- **P6. Breadth is kept, as modules** (§5.2).

The gates are the tests. S1 to S4 (security), A1 to A4 (artificial intelligence) and Q1 to Q4 (quantum computing) are defined in Amendment A-1 §9; G0 and G1, which every target must pass, in Amendment A-2 §6. A statement about Dhar's security, speed or correctness, or about a target, names its gate or is worded as an aim (Tempering Dhar §11).

6. Language Preview

Dhar defaults to Python-like concision, with explicit typing available for systems-level control — both forms are first-class and coexist in the same program.

```
# concise, inferred -- unqualified bindings are mutable by default
name = "Dhar"
count = 10
count = count + 1 # fine: count was never locked
# explicit, for systems-level code or to opt into immutability
lock pi: f64 = 3.14159
flux total: i32 = 0
address: raw[4096]
# collections, inferred (Spec1 §13)
scores = [98, 87, 92] # a list
lock bonus = (n: i32) -> n + 5 # a closure -- functions are first-class
each s in scores:
    gen(bonus(s))
# structs and methods -- brace-delimited, unlike the rest of the language
mold Point {
    flux x: i32
    flux y: i32
}
forge Point {
    task init(a: i32, b: i32):
        x = a
        y = b
}
task dist(p: view Point): i32 # view = read-only reference
    give p.x + p.y
# modules, entry point, and error handling
pull math
expose dist
task core(): # core() is the standard entry point
    flux s: state = may_fail()
    trap s
    enforce s == 0
```

This is a taste, not the specification. Every keyword shown above — the full keyword set (including generics, sum types, interfaces, and async), the type system, the memory model, and collections — is ratified in Spec1. `gen` is the portable print function (Spec1 §20.2): it writes exactly the text of its value and nothing more, on every committed target.

7. Positioning: How Dhar Compares

LANGUAGE	PERFORMANCE	MEMORY SAFETY	TOOLCHAIN INDEPENDENCE	MULTI-PARADIGM DEPTH
C	Near-hardware	Manual, unchecked	Depends on GCC/Clang	Procedural only
C++	Near-hardware	Manual + optional RAII	Depends on GCC/Clang/MSVC	Procedural + OOP + generics
Rust	Near-C	Compile-time ownership	Compiler self-hosted; default code generator is LLVM	Mostly procedural/OOP
Java	JIT, GC pauses	Managed (GC)	Depends on the JVM	OOP-centric

LANGUAGE	PERFORMANCE	MEMORY SAFETY	TOOLCHAIN INDEPENDENCE	MULTI-PARADIGM DEPTH
Python	Interpreted	Managed (GC)	Depends on the CPython runtime	Dynamic, not symbolic-native
Go	Compiled, GC pauses	Managed (GC)	Self-hosted, own backend	Procedural/OOP-lite
Zig	Near-C	Manual + compiler checks	Frontend independent; relies on LLVM for optimized code, with back ends of its own in development	Procedural
Dhar	Near-hardware — the aim is at least C-class, measured at gate A4	Ownership/arena + bounds checks on every index (gate S1)	Self-hosting from Stage-beta, custom IR, no LLVM/JVM (P1)	Procedural, functional, OOP, logical, symbolic — plus optional dynamic typing

Dhar's row states this Canon's design commitments, deliberately placed in the same performance tier as C rather than merely "close to" it, each with the gate that tests it (Amendment A-1 §9). Stage-gamma (Optimizing Dhar) carries the performance commitment; Spec1 carries the safety and paradigm commitments; Stage-alpha and Stage-beta carry the independence commitment. The rows for other languages summarise their publicly documented design as context; they are not claims of this Canon, and they are checked against each project's own documentation whenever this table is revised.

8. The Five Stages

STAGE	NAME	MANDATE
Stage-alpha	Forge of Dhar	Bootstrap compiler proving the pipeline end-to-end for a minimal language subset, on every committed target (§4.1)
Stage-beta	Honing Dhar	A complete compiler written entirely in Dhar, capable of compiling itself
Stage-gamma	Optimizing Dhar	SSA, register allocation, jump tables, vectorization
Stage-delta	Gearing Dhar	Standard library and tooling: build, test, format, packages, LSP, and the domain modules of §5.2
Stage-epsilon	Deploying Dhar	Cross-compilation, debug info, fuzzing, reproducible builds, SBOM, signing, release gates, hosted services

Stage-beta begins only when Stage-alpha has passed gate G1 on every committed target (Amendment A-2 §2). Stage-gamma must not begin until Stage-beta is genuinely self-hosting and the syntax ratified in Spec1 is stable. The work of the three tracks begins only when Stage-beta compiles itself (§5.1). Full mandates, acceptance criteria, and required tests for each stage are specified in their own Volume II documents.

9. Reading the Rest of the Canon

- Writing Dhar code or building the compiler → Spec1, then the relevant Volume II stage document, and Tooling Dhar for diagnostics, editor support and the in-browser playground.
- Evaluating Dhar for adoption, funding, or partnership → Casting Dhar and Fueling Dhar.
- Assessing Dhar for security, critical-infrastructure, or defence use → Tempering Dhar, alongside Spec1 and Amendment A-1.
- Joining as a contributor → The Guild of Dhar.
- Tracking overall progress → The Pact of Developing Dhar; what exists is reported in the Progress Reports, which are outside the Canon.
- Understanding what changed and why → Amendments A-1 (Direction) and A-2 (Stage 0 Targets), and the list of amended documents in the Master Index §8.2.

The full index lives in The Canon of Dhar — Master Index. If Linux is the proof that an independently built, freely extensible foundation can outgrow the proprietary systems around it, this Canon is Dhar's plan for the same outcome one layer up the stack, at the language itself.

Revision log

Spec0 — Overview & Vision · v1.3 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0	September 7, 2026	rewritten as a pure forward-looking specification. v1.1 — fixed mold/forge to brace-delimited syntax for consistency with Spec1; unqualified bindings now documented as mutable (flux) by default; performance positioning restated as near-hardware/C-class rather than merely near-C; keyword count updated to 38 following the addition of wild (Spec1 §4.4); added the Linux analogy. v1.2 — added §0, Philosophy & Naming, grounding Dhar and Kun in researched etymology; keyword count updated to 46 following Spec1 v3.0.
v1.3	October 26, 2026	Amendments A-1 and A-2: the direction (§1, §5.1), breadth kept as modules (§5.2), the principles and gates (§5.3), the targets (§4.1), the stage plan (§8) and the reading list (§9); statements about other languages (§2.2, §7), the intermediate representation (§3, §4) and Spec1 references (§6) corrected.